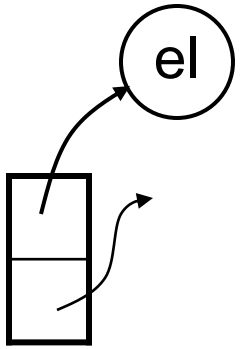
Linked lists

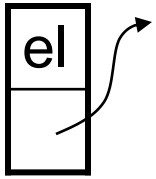
- Singly linked lists
- Doubly linked lists
- Circular lists
- Self-organizing lists
- `LinkedList` and `ArrayList`

Singly linked list node

```
public class SLLNode<T> {
    public T info;
    public SLLNode<T> next;
    public SLLNode() {
        next = null;
    }
    public SLLNode(T el) {
        info = el; next = null;
    }
    public SLLNode(T el, SLLNode<T> p) {
        info = el; next = p;
    }
}
```

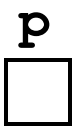


abbreviated as:

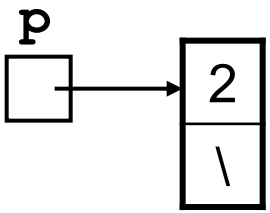


Singly linked list

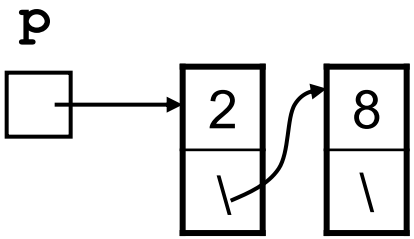
```
SLLNode<Integer> p;
```



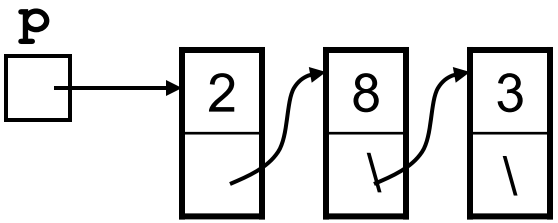

```
p = new SLLNode<Integer>(2);
```




```
p.next =
new SLLNode<Integer>(8);
```

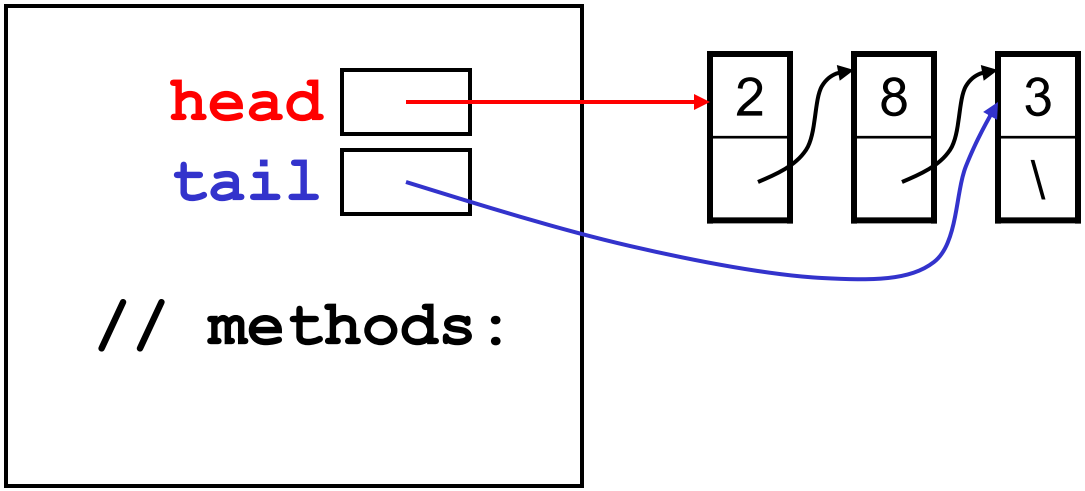



```
p.next.next =
new SLLNode<Integer>(3);
```

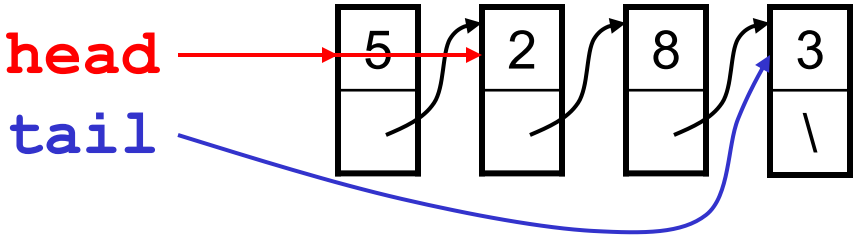


Singly linked list

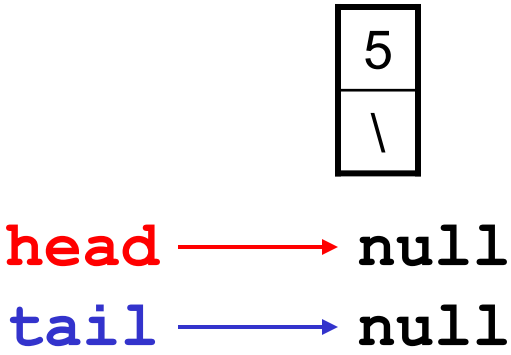
```
public class SLL<T> {
    protected SLLNode<T> head, tail;
    // methods:
    ...........
}
```



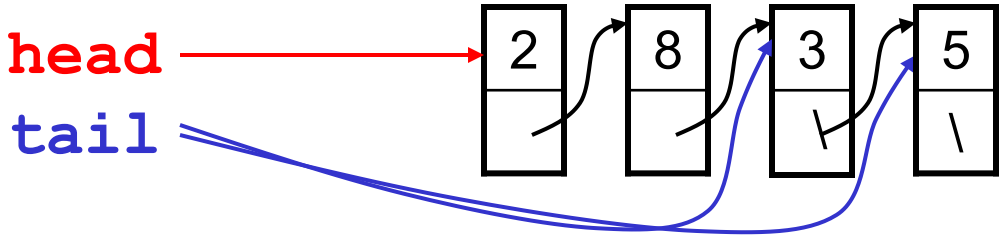
Singly linked list: insertion at the head



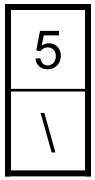
special case:

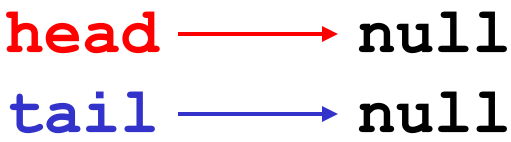


Singly linked list: insertion at the tail

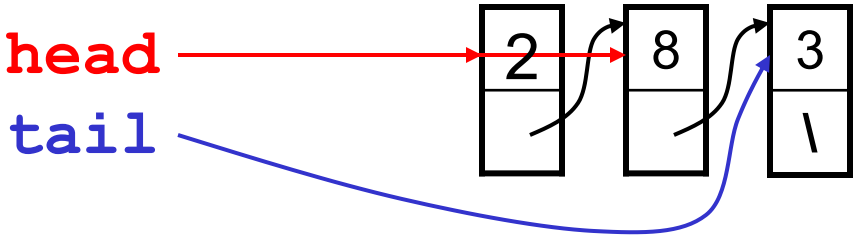


special case:

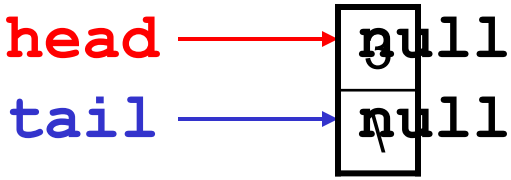




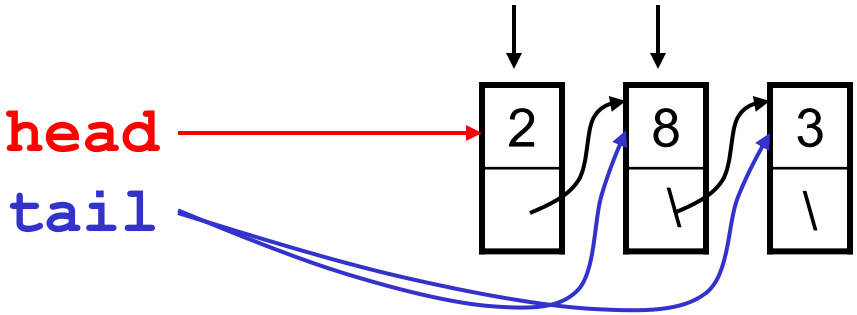
Singly linked list: deletion from the head



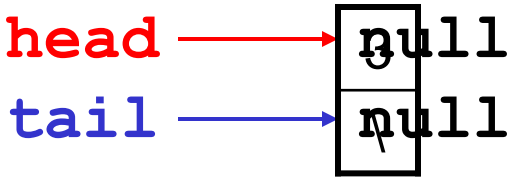
special case:



Singly linked list: deletion from the tail

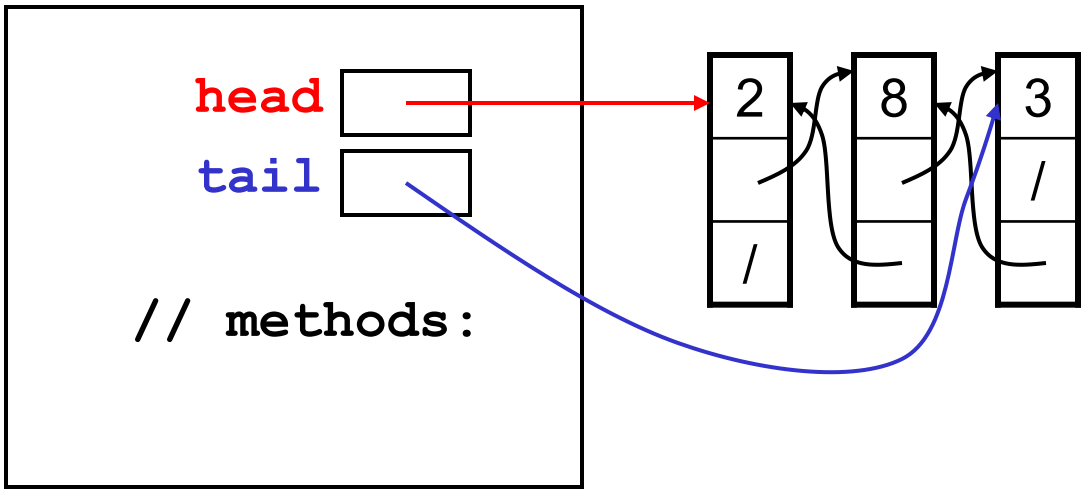


special case:

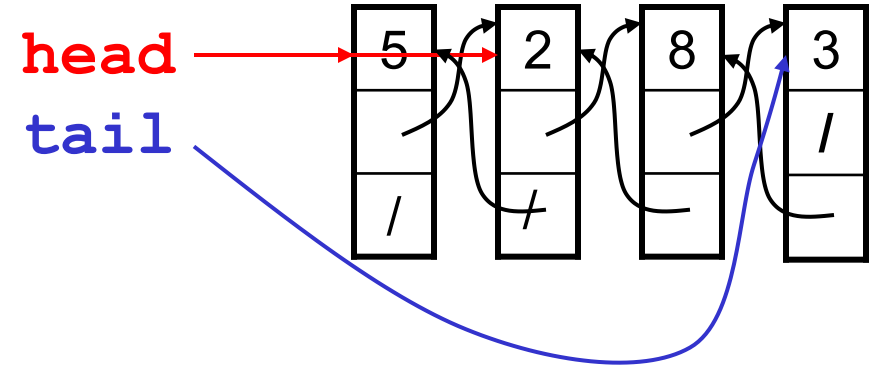


Doubly linked list

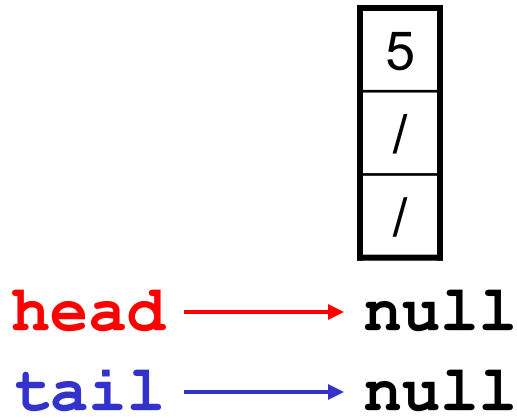
```
public class DLLNode<T> {
    public T info;
    public DLLNode<T> next, prev;
    // constructors:
    . . . . . . . . . . . .
}
public class DLL<T> {
    protected DLLNode<T> head, tail;
    // methods:
    . . . . . . . . . . .
}
```



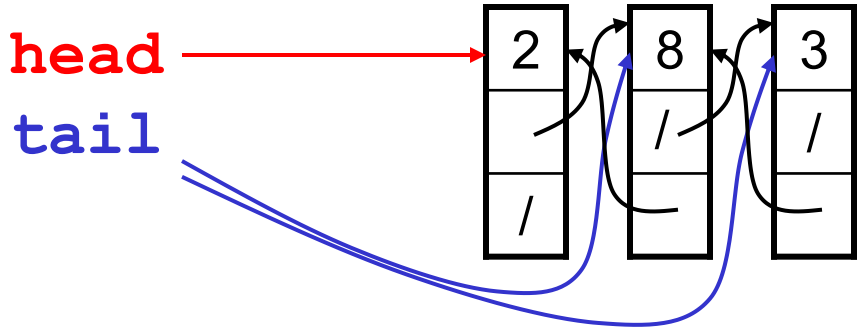
Doubly linked list: insertion at the head



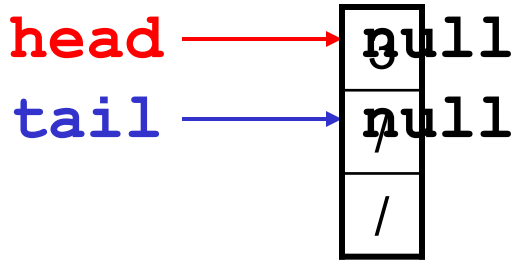
special case:



Doubly linked list: deletion from the tail

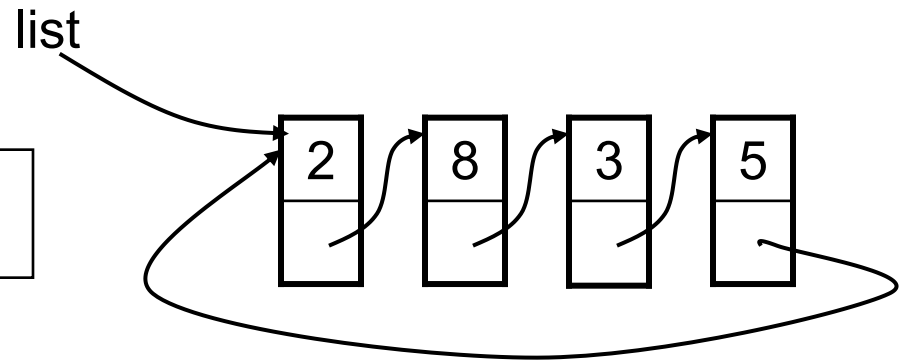


special case:

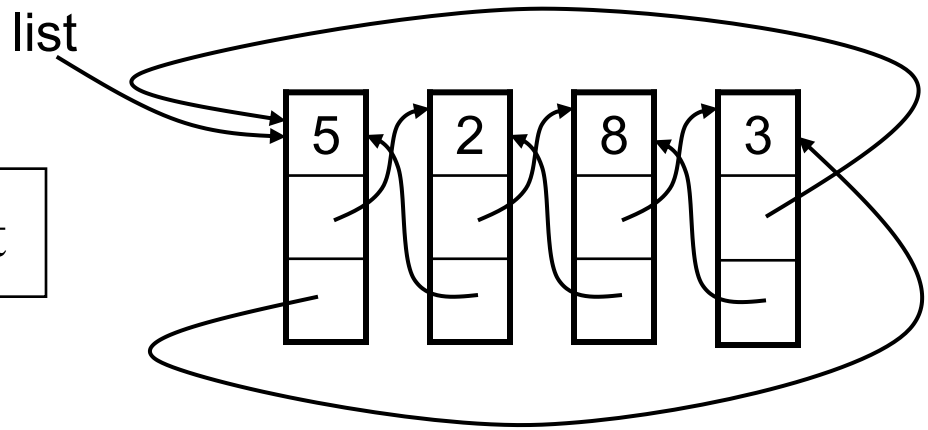


Circular linked lists

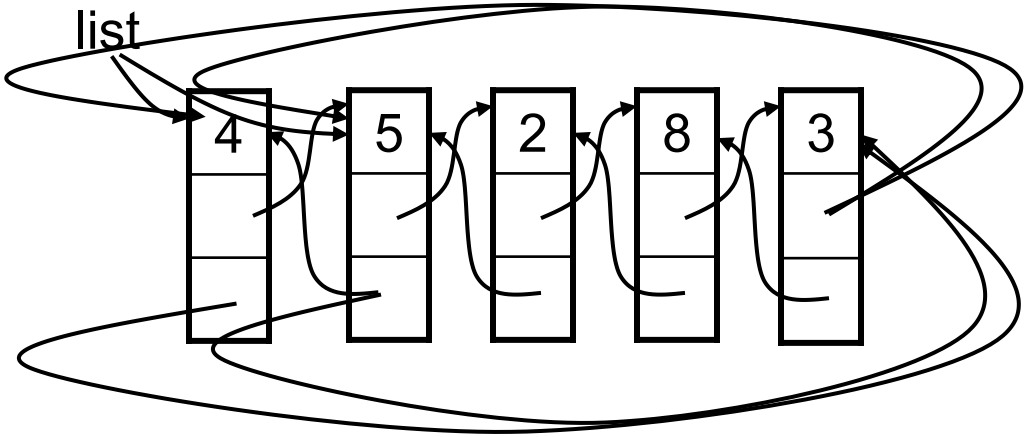
Circular singly linked list



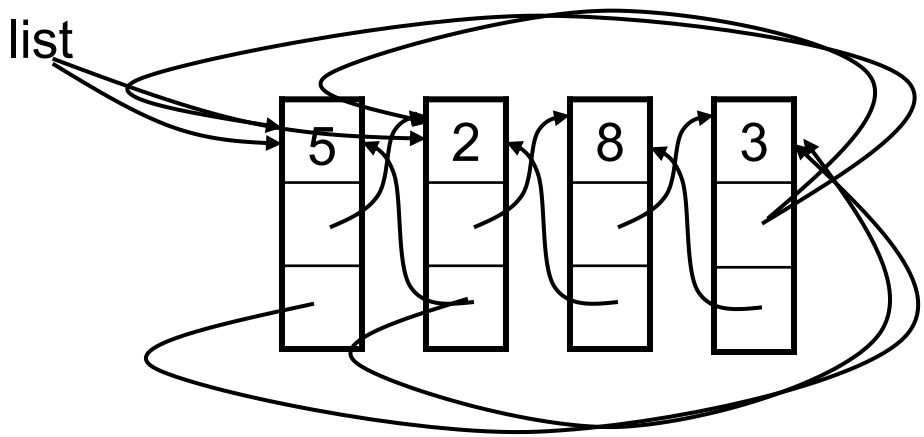
Circular doubly linked list



Circular doubly-linked list: insertion

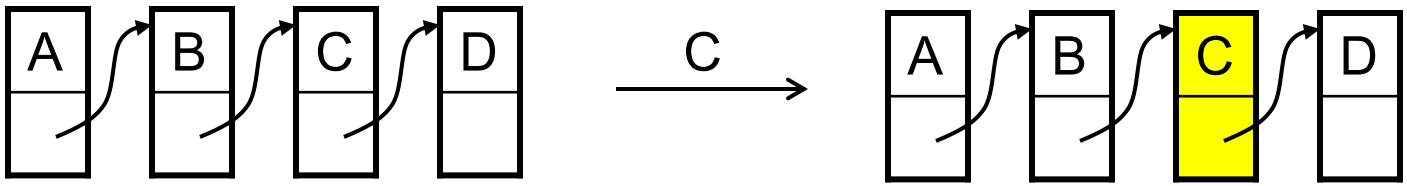


Circular doubly-linked list: deletion

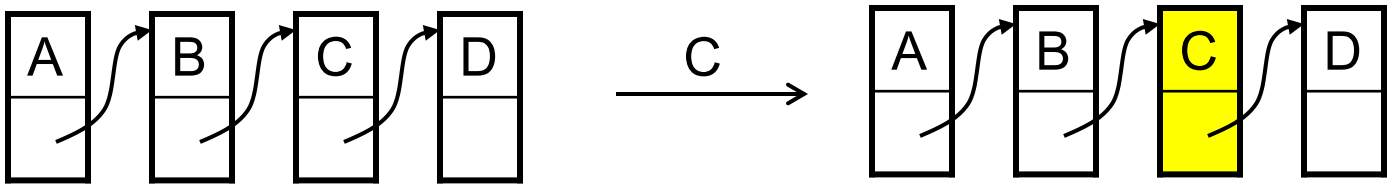


Self-organizing lists

Move-to-front method: after the desired element is located, put it at the beginning of the list.

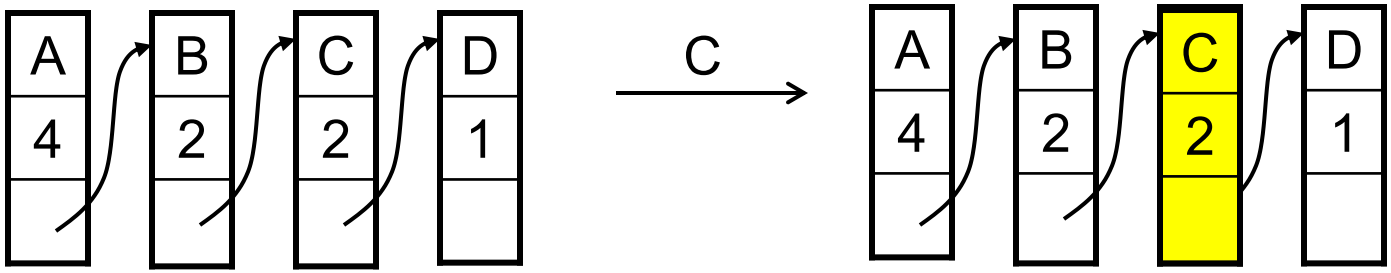


Transpose method: after the desired element is located, swap it with its predecessor.

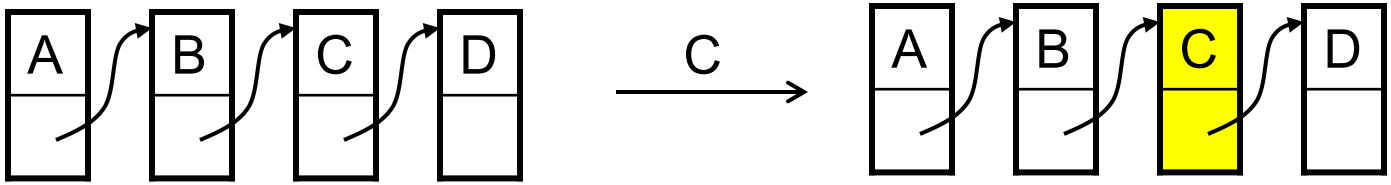


Self-organizing lists, cont'd

Count method: order the list by the number of times elements are being accessed.

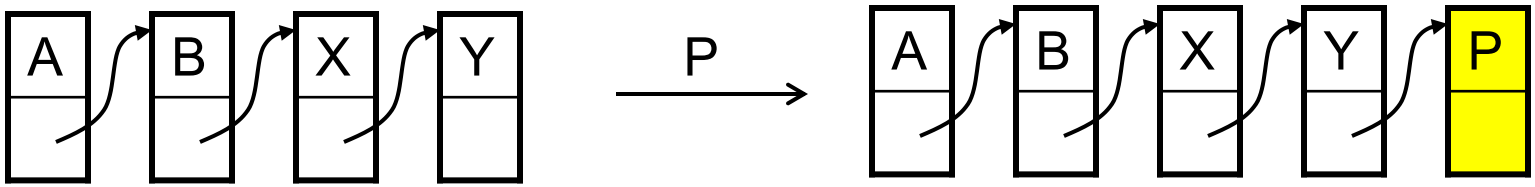


Ordering method: order the list using a criterion natural for information under scrutiny.

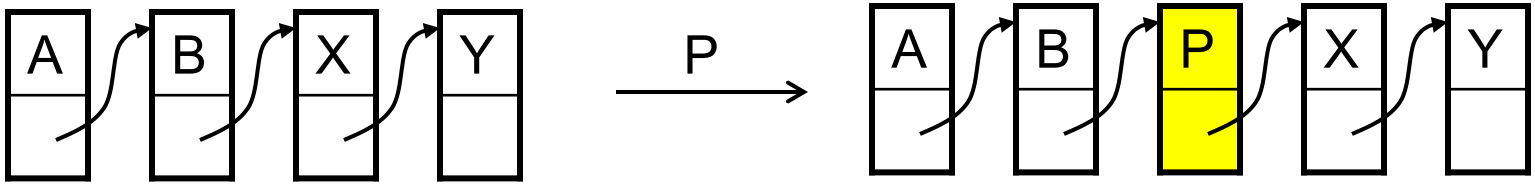


Self-organizing lists: accessing elements for the first time

move-to-front, transpose, and count methods:



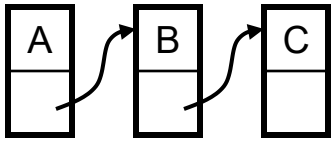
ordering method:



Self-organizing lists: example

element	plain	move-to-front	trans-pose	count	ordering
D	D	D	D	D_1	D
A	DA	DA	DA	D_1A_1	AD
C	DAC	DAC	DAC	$D_1A_1C_1$	ACD
C	DAC	CDA	DCA	$C_2D_1A_1$	ACD
B	DACB	CDAB	DCAB	$C_2D_1A_1B_1$	ABCD
A	DACB	ACDB	DACB	$C_2A_2D_1B_1$	ABCD
A	DACB	ACDB	ADCB	$A_3C_2D_1B_1$	ABCD

ABC represents the list



Self-organizing lists: performance

Method	Different words/all words 6835/93087
optimal	7.6
plain (head)	86.6
plain (tail)	19.3
move to front (head)	15.0
move to front (tail)	14.3
transpose (head)	75.4
transpose (tail)	18.2
count	13.4
alphabetical order	54.9

Priority queue: linked list implementation

number of operations	enqueue	dequeue
unordered list	const	n
ordered list	$\leq n$	const